Cesar Giner
Program 2 - AI Incorporated Code

A:
Token structure

CAn we use a struct and what would that look likde, you can output code

Yes. In Racket, using `struct` makes tokens explicit and type-safe. You can define a general `token` struct with fields for type and value. Example:

racket

```
#lang racket

;; Define a token struct
(struct token (type value) #:transparent)

;; Examples of tokens
(define id-token (token 'IDENT "myVar"))
```

B:
'Make-number-token' function, several errors here, had to debug

racket

```
#lang racket

(struct token (type value) #:transparent)

(define (make-number-token num-str)
  ;; Remove leading/trailing whitespace just in case
  (define s (string-trim num-str))

  (cond
    ;; Check if it matches a floating-point pattern
    [(regexp-match? #px"^[0-9]+\\.[0-9]+$" s)
     (token 'FLOAT_LIT (string->number s))]  ; convert to number

    ;; Check if it matches an integer pattern
    [(regexp-match? #px"^[0-9]+$" s)
     (token 'INT_LIT (string->number s))]

    ;; Otherwise, invalid numeric literal
    [else
     (error "Lexical Error: invalid number literal" s)]))
```

C:
‘Peek’ function

2. Each function returns a pair: `(parsed-subtree remaining-tokens)`.

`peek`

- Returns the **first token** of the list without modifying the stream.
- Returns `#f` if the token list is empty.

racket

```
(define (peek tokens)
  (if (null? tokens)
      #f
      (car tokens)))
```

---

consume

D:
Asked LLM to generate some tests for me

all in one block of code

Here’s a **single Racket block** that defines and runs a suite of tests for your `RDP` parser. It prints the input, expected AST, and actual output.

racket

```
#lang racket

;; Assume RDP is already defined:
;; (define (RDP source) (parse-program (scan source)))

(define test-cases
  (list
   ;; Simple literals
   (list "42" 42)
   (list "3.14" 3.14)
   (list "x" 'x)

   ;; Unary operators
   (list "-42" '(neg 42))
   (list "-x" '(neg x))

   ;; Binary operators
   (list "1 + 2" '(+ 1 2))
   (list "x - y" '(- x y))

   ;; Operator precedence
   (list "2 + 3 * 4" '(+ 2 (* 3 4)))
```